EngageIQ: Smart Engagement Opportunity Scorer

Technical Brief | Big Data (BAX 423) Final Project

Author: Saurav Kanegaonkar | **Student ID:** 924465543

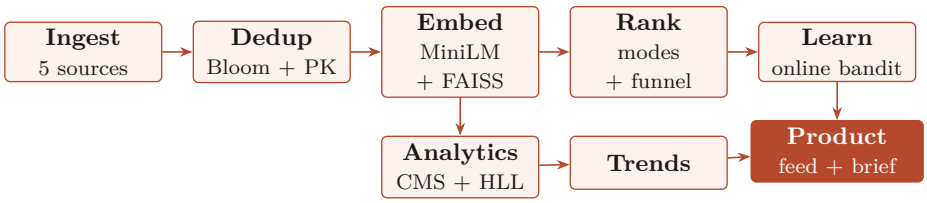
Live application: engageiq-148810228333.us-central1.run.app

Submitted materials:

- `code/` (single-command FastAPI app)
- `data/` (offline 20,995-record SQLite snapshot)
- `brief.pdf` (this technical brief)
- `prompts.md` (the AI prompt log)
- `README.md` (overview and run instructions)

1. System Architecture

EngageIQ reframes the problem as an attention-portfolio recommender: it treats a unit of a professional's time as the item being allocated, and ranks opportunities by expected impact per unit of effort against a weekly budget. The engine is one linear pipeline with a parallel batch-analytics branch, wrapped in a product rather than a notebook.



Five source adapters normalize to one canonical `Opportunity` schema and pass a two-layer deduplicator: a from-scratch Bloom filter (Lecture 2) backed by a database primary key. Two SQLite databases stay separate: a read-only corpus (`engageiq.sqlite`, the snapshot a grader runs with no API keys) and a read-write user-state database (`engage.sqlite`: accounts, profiles, interactions, an event stream, and the learner weights). The product surfaces the result as a ranked feed with a plain-language "Why this", an LLM-drafted suggested action, a Trends tab, and an exportable brief (Capability 6). The whole app is one FastAPI container on Google Cloud Run with both transformer models baked in, scaling to zero when idle. The offline snapshot holds 20,995 deduplicated, 2026-only records across all 15 required technical domains:

Dev.to	6,607	Reddit	3,768
Bluesky	5,482	GitHub	1,000
Hacker News	4,138	**Total**	**20,995**

2. BAX-423 Techniques and Why

Four course technique families drive the pipeline. Each was chosen against a cheaper baseline and benchmarked; for each, where it is used and why.

Embeddings and ANN retrieval (Lecture 5)

Where. Opportunity text and the user's interest profile are encoded with Sentence-BERT (`all-MiniLM-L6-v2`, 384-d); retrieval is exact cosine over an L2-normalized FAISS `IndexFlatIP`. This is the matching core.

Why. The corpus is short, jargon-dense Reddit topics and GitHub issues where semantic context beats keyword overlap. MiniLM was chosen over `bge-small` for cleaner domain separation. Retrieval precision@10 is 0.89, robust to noisy domain labels.

Multi-stage ranking (Lecture 7)

Where. A candidate-generation to re-ranking funnel: stratified per-source FAISS recall, hard filters (platform, domain, language), a cross-encoder relevance re-rank, a goal-weighted impact score, MMR for diversity, and a time-budget knapsack that fills the weekly hours. Four engagement modes (Contribute, Discuss, Monitor, Promote) set the signal weights and the action verb.

Why. Dense embeddings alone lose to keyword search on this jargon corpus (Section 3); reading the query and document jointly with a cross-encoder recovers the gap, and the knapsack turns scores into a realistic weekly plan.

Probabilistic sketches (Lecture 2)

Where. The batch-analytics layer over the full corpus: a from-scratch Count-Min Sketch (term frequencies) and HyperLogLog (distinct contributors) power the Trends tab; a Bloom filter gates ingestion dedup.

Why. Both merge without rescanning, which is what lets the dashboard roll a persona's domains up on the fly, at a fraction of the memory: 1.16% max error at 9.8x less memory (CMS) and 0.43% at 48.7x smaller (HLL) versus exact counts.

Adaptive learning (Lecture 8)

Where. A per-(user, persona) online weight-learner (Widrow-Hoff over the nine scoring signals) updates on every engage, skip, or save and persists across reloads.

Why. It personalizes ranking from real feedback, warm-started from the mode defaults and renormalized so the weights stay interpretable. Over 60 simulated rounds, nDCG@5 rose from 0.679 to 0.960 (Section 3).

3. Matching and Ranking Benchmarks

The Stage-2 re-ranker was benchmarked with an LLM-as-judge harness (relevance 0 to 3, pooled and cached for reproducibility), reporting nDCG@10 and P@10.

Stage-2 matching strategy	nDCG@10
Dense (MiniLM cosine, alone)	0.63
TF-IDF (keyword)	0.77
Hybrid (dense + keyword)	0.70
Cross-encoder re-rank (chosen)	0.76

The honest finding: dense embeddings alone (0.63) underperform plain TF-IDF (0.77) on this short,

jargon-heavy corpus. The design treats this as the reason for the cross-encoder Stage-2 (0.76), which matches keyword quality. The full assembled pipeline (recall, filters, cross-encoder, impact, MMR, knapsack) reaches a mean nDCG@10 of 0.711 across the four test personas. Adaptive learning was validated over 60 feedback rounds (nDCG@5 0.679 to 0.960). A 53-check automated suite covers the sketches, analytics, and the trends and brief endpoints; all pass.

4. Test Persona Results

Each of the four provided personas was evaluated against its pass criteria and each of the six core capabilities. All twelve persona pass-criteria pass; the capability coverage is below.

Core capability	Sofia	David	Lina	Raj
1. Ingestion + dedup + storage	Pass	Pass	Pass	Pass
2. Embedding + similarity retrieval	Pass	Pass	Pass	Pass
3. Scoring + multi-stage ranking	Pass	Pass	Pass	Pass
4. Adaptive learning from feedback	Pass	Pass	Pass	Pass
5. Batch analytics + trend detection	Pass	Pass	Pass	Pass
6. Dashboard + Why-this + brief	Pass	Pass	Pass	Pass

The persona-specific criteria are met by the goal-conditioned modes. Sofia (Contribute) gets beginner-friendly GitHub issues with C++/Rust excluded and a contribution-worthiness gate; David (Discuss) gets Kubernetes and infra threads, not general web dev, plus a standout-repo lane (high activity, few contributors); Lina (Monitor) gets a velocity radar ranked by recency and star/comment surge; Raj (Promote) gets developer-tools threads, and the learner deprioritizes low-engagement ones after skips.

5. Limitations

Snapshot recency bias. The offline scrape skews to recent weeks (a collection artifact), so a raw volume-over-time line would mislead. Trend signals therefore use each domain's share of weekly activity on complete weeks only, which is robust to the bias, and the limitation is stated in-product.

Domain-label precision. Labels began as keyword matches (about 62% precision). A multi-label re-classification with an independent verification gate lifted this to 78%, but several niche domains (for example Python data engineering, AI research) remain weaker, so a small fraction of items carry an imperfect domain tag.

GitHub supply. GitHub is the smallest source (1,000 records), so a Contribute-heavy persona can see fewer GitHub items than its plan requests even though the matching is correct; the candidates exist in the corpus but do not always rank into the top slice. This is a data-supply limit, not a ranking bug.

Hosting trade-offs. The free Cloud Run instance has an ephemeral filesystem, so runtime user accounts and learned weights reset on a redeploy or a cold start; the read-only corpus and all ranking are unaffected. Durable accounts would need a managed database, which was out of scope for a free single-instance deployment.